

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

[Полное наименование образовательной организации]

Кафедра [наименование кафедры]

ОТЧЁТ

по лабораторной работе №2 по дисциплине «Защита информации»

Алгоритм шифрования RSA

Выполнил: студент группы [группа]
[Фамилия И. О.]

Проверил: [должность]
[Фамилия И. О.]

[Город] — 2026

1 Цель работы

Изучить и проанализировать работу криптографического алгоритма шифрования RSA. Написать программу, реализующую работу протокола.

Разработана программа `rsa.py` (Python 3) с режимами генерации случайных ключей, ручного ввода ключей, шифрования и расшифрования текста ASCII, самопроверки и вычисления закрытого ключа по открытому.

2 Теоретические сведения

Симметричные криптосистемы используют один секретный ключ, асимметричные — пару: открытым шифруют, закрытым расшифровывают. RSA — асимметричная.

Генерация ключей: выбираются большие различные простые p , q ; вычисляются $n = p \cdot q$ и $\phi = (p-1)(q-1)$; случайно выбирается d , взаимно простое с ϕ ; e находится из $e \cdot d \bmod \phi = 1$. Открытый ключ — $\{e, n\}$, закрытый — $\{d, n\}$. При $p = q$ формула для ϕ неверна.

Текст разбивается на блоки $0 \leq M_i < n$; шифрование открытым ключом, расшифрование закрытым:

$$C_i = M_i^e \bmod n, M_i = C_i^d \bmod n.$$

Корректность следует из $e \cdot d = 1 + k\phi$ и теоремы Эйлера $M^\phi \equiv 1 \pmod n$ при $\text{НОД}(M, n) = 1$:

$$C^d = M^{ed} = M \cdot (M^\phi)^k \equiv M \pmod n.$$

Для M , кратных p или q , сравнение доказывается по модулям p и q и объединяется китайской теоремой об остатках. Эталон самопроверки (`self_test`) — пример методички: $n = 33$, $d = 3$, $e = 7$; блоки $3, 1, 2 \rightarrow 9, 1, 29$.

Зная p и q , противник вычисляет ϕ и $d = e^{-1} \bmod \phi$; стойкость RSA определяется трудностью факторизации n . Вспомогательные алгоритмы: `modpow`, расширенный Евклид (`egcd`, `modinv`), пробное деление (`is_prime`, `factor`).

3 Постановка задачи

Задание методички: «Написать на языке высокого уровня программу, которая бы реализовывала алгоритм шифрования/расшифрования RSA. Программа должна поддерживать как ручной ввод входных значений, так и случайный. Программа должна работать с любыми символами из кодировки ASCII». Язык реализации — Python 3, стандартная библиотека.

Из задания вытекают требования: **T1** — ключи вычисляются по методичке (d случайное, e из $e \cdot d \bmod \varphi = 1$), пример $p = 3$, $q = 11$ воспроизводится; **T2** — текст шифруется открытым ключом $\{e, n\}$ и расшифровывается закрытым $\{d, n\}$; **T3** — ключи вводятся вручную с проверкой условий и сообщением об ошибке; **T4** — ключи генерируются случайно; **T5** — принимаются любые символы ASCII, проверяется прогоном всех кодов.

Принятые допущения:

- 1) Единица шифрования — один символ: блок $M = \text{ord}(ch)$, многосимвольные блоки не используются.
- 2) Порог $n \geq 256$ (N_MIN) взят с запасом: программа принимает ASCII 0..127 (`read_text`), для которого хватило бы $n \geq 128$; минимальный допустимый $n = 259$ ($7 \cdot 37$), в самопроверке используется $5 \cdot 53 = 265$.
- 3) «Большие простые» трактуются как простые из $[100, 9999]$ (P_MIN , P_MAX) при генерации; простота проверяется пробным делением до $\sqrt{p}$ (`is_prime`), вероятностные тесты не применяются.
- 4) Случайно генерируются только параметры ключа (p , q , d ; e вычисляется); открытый текст во всех режимах вводит пользователь.
- 5) Шифртекст — список целых чисел через пробел; расшифрование — по $\{d, n\}$ без p и q .
- 6) Реализуется учебный RSA без паддинга: одинаковые символы дают одинаковые C_i ; известное ограничение, а не дефект.

4 Описание алгоритма

Схема алгоритма приведена на рисунке 4.1: получатель формирует пару ключей и публикует $\{e, n\}$; отправитель шифрует каждый символ открытым ключом и передаёт список чисел; получатель восстанавливает текст закрытым ключом $\{d, n\}$.

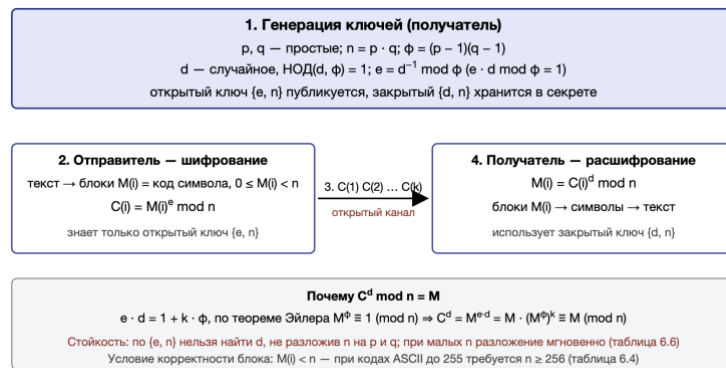


Рисунок 4.1 — Схема RSA: генерация ключей, шифрование открытым и расшифрование закрытым ключом

Генерация ключей реализована функцией `make_keys`; `key_errors` отвергает непростые p, q , $p = q$ и $n = p \cdot q < 256$ (`N_MIN`); `modinv` находит $e = d^{-1} \bmod \phi$ расширенным алгоритмом Евклида (`egcd`) и нормализует результат в $[1, \phi-1]$.

Шифрование посимвольно: `encrypt_text` берёт для каждого символа $M = \text{ord}(\text{символа})$, `encrypt_int` проверяет $0 \leq M < n$ и вычисляет $C = M^e \bmod n$; шифртекст — список чисел. `decrypt_text` восстанавливает $M = C^d \bmod n$ (`decrypt_int`) по $\{d, n\}$ без p и q ; блок, не являющийся кодом символа (неверный ключ, чужой шифртекст), вызывает `ValueError`.

Возведение в степень выполняет `modpow` бинарным методом: показатель просматривается по битам, основание возводится в квадрат, при единичном бите результат умножается на основание, после каждого умножения берётся остаток по модулю. Вычислять M^e целиком нельзя: при четырёхзначных p, q это число из миллионов цифр. Метод опирается на тождество

$$(a \cdot b) \bmod n = ((a \bmod n) \cdot (b \bmod n)) \bmod n,$$

поэтому промежуточные значения не превышают n^2 , а умножений — около $2 \cdot \log_2 e$. Восстановление закрытого ключа по открытому (`recover_private_key`): `factor` раскладывает $n = p \cdot q$ пробным делением до $\sqrt{n}$, затем $\phi = (p-1)(q-1)$ и $d = e^{-1} \bmod \phi$ (`modinv`) — как при генерации.

5 Описание программы

Программа `rsa.py` написана на Python 3 (стандартная библиотека), запуск — `python3 rsa.py`.

Состав: `modpow`, `is_prime`, `gen_prime`, `gcd`, `egcd`, `modinv`; `key_errors`, `make_keys`; `encrypt_int`, `decrypt_int`, `encrypt_text`, `decrypt_text`; `self_test`, `factor`, `recover_private_key`; **ВВОД** — `read_int`, `read_text`; класс `State` хранит ключи и последний шифртекст; **режимы** `random_keys`, `manual_keys`, `encrypt_mode`, `decrypt_mode`, `attack_mode`; **ЦИКЛ МЕНЮ** — `main`.

Меню: 1 — случайные ключи (простые p, q из $[100, 9999]$, случайное d с $\text{НОД}(d, \phi) = 1$, $e = \text{modinv}(d, \phi)$); 2 — ввод p, q, d вручную; 3 — шифрование текста ASCII; 4 — расшифрование по $\{d, n\}$ без p, q (Enter — текущий ключ и последний шифртекст); 5 — самопроверка; 6 — вычисление закрытого ключа по открытому (`factor` → ϕ → d); 0 — выход.

При ручном вводе `read_int` повторяет запрос после отказа:

- «Нужно ввести целое число.»;
- « $p = 9$ не простое»;
- « q должно отличаться от p : ...»;
- « $n = p \cdot q = 33 < 256$: ...»;
- « $\text{НОД}(d, \phi) = 4 \neq 1$: обратного e нет».

Шифртекст — числа через пробел, каждое из $[0, n-1]$. При неверном ключе или чужом шифртексте блок M не является кодом символа: `decrypt_text` возбуждает `ValueError`, пункт 4 печатает «Не удалось расшифровать: ...». EOF или Ctrl+C завершают программу сообщением «Выход.».

6 Результаты тестирования

Пример методички воспроизведён самопроверкой (пункт 5, `self_test`); строки с кодами ASCII получены прямым вызовом `modpow` из интерпретатора, поскольку `encrypt_int` отвергает $M \geq n$.

Таблица 6.1 — Пример методички ($p = 3$, $q = 11$, $n = 33$, $d = 3$, $e = 7$) на числах $A = 1$, $B = 2$, $C = 3$ и на кодах ASCII

Блок М	С ожидаемый	С полученный	М' после расшифрования	Совпало
$C = 3$	9	9	3	да
$A = 1$	1	1	1	да
$B = 2$	29	29	2	да
'C' = 67 (ASCII)	—	1	$1 = 67 \bmod 33$	НЕТ ($M \geq n$)
'A' = 65 (ASCII)	—	32	$32 = 65 \bmod 33$	НЕТ ($M \geq n$)
'B' = 66 (ASCII)	—	0	$0 = 66 \bmod 33$	НЕТ ($M \geq n$)

Блоки 3, 1, 2 дали $C = 9, 1, 29$ и вернулись без искажений; коды 67, 65, 66 при $n = 33$ после расшифрования дали 1, 32, $0 = M \bmod 33$ ($M \geq n$). Экран самопроверки — рисунок 6.2.

```
Лабораторная работа №2. Алгоритм шифрования RSA
1 - случайные ключи
2 - ввод ключей вручную (p, q, d)
3 - зашифровать текст открытым ключом {e, n}
4 - расшифровать закрытым ключом {d, n}
5 - самопроверка (пример методички, все коды ASCII)
6 - вычислить закрытый ключ по открытому (факторизация n)
0 - выход
Выбор: 5
Пример методички: p=3, q=11, n=33, φ=20, d=3 → e=7 (ожидалось 7)
M = [3, 1, 2] → C = [9, 1, 29] (ожидалось [9, 1, 29]) → M' = [3, 1, 2]:
PASS
Все коды 0..255 через ключ n=265 (p=5, q=53, d=7, e=119): совпало 256/256:
PASS
Итог самопроверки: OK
```

Рисунок 6.2 — Самопроверка: пример методички и все коды ASCII

Итог «ОК»: 256/256 кодов через ключ $5 \cdot 53 = 265$. Ручной ввод (пункт 2, `manual_keys`) проверен на восьми наборах.

Таблица 6.2 — Ручной ввод ключей (пункт меню 2) и реакция на некорректные значения

№	Ввод	Реакция программы
1	$p = 5, q = 53, d = 7$	принято: $n = 265, e = 119$; цикл шифрование→расшифрование пройден
2	$p = 61, q = 53, d = 17$	принято: $n = 3233, e = 2753$; цикл шифрование→расшифрование пройден
3	$p = 9$ (составное)	сообщение « $p = 9$ не простое...», повторный запрос
4	$q = p = 11$	сообщение « q должно отличаться от p ...», повторный запрос
5	$p = 3, q = 11$ ($n = 33 < 256$)	сообщение « $n = p \cdot q = 33 < 256$...», повторный запрос
6	$p = 3, q = 43$ ($n = 129 < 256$)	сообщение « $n = p \cdot q = 129 < 256$...»,

№	Ввод	Реакция программы
		повторный запрос
7	$p = 61, q = 53, d = 4$ ($\text{НОД}(d, \varphi) \neq 1$)	сообщение « $\text{НОД}(d, \varphi) = 4 \neq 1 \dots$ », повторный запрос
8	$d = \text{«abc»}$	сообщение «Нужно ввести целое число...», повторный запрос

Корректные пары приняты ($n = 265, e = 119; n = 3233, e = 2753$) и прошли цикл шифрование→расшифрование; составное $p = 9, q = p, n = 33$ и $129, \text{НОД}(d, \varphi) = 4$, нечисловое abc отвергнуты с повторным запросом.

Случайные ключи (пункт 1, `random_keys`) проверены на четырёх текстах, включая все печатные символы 32–126 и пустую строку.

Таблица 6.3 — Шифрование и расшифрование при случайных ключах

№	Текст	p, q	d, e	Начало шифртекста	Верно
1	«Hello, RSA 2026!» — буквы, цифры, знаки, пробел	6991, 6143	39306859, 25124419	10947667 10311295 1210064 1210064 ...	да
2	«aaa bbb» — повторы символов	4337, 7951	31501307, 2504243	6333550 6333550 6333550 24314473 ...	да
3	все печатные символы 32–126 (95 симв.)	4507, 251	69437, 763373	593251 109247 754828 849225 ...	да
4	«» — пустая строка	7607, 9239	4512395, 1685571	—	да

Все тексты расшифрованы верно; одинаковые символы дают одинаковые $C(i)$ (6333550 для «а»). Полный сеанс (пункты 1, 3, 4, 6) — рисунок 6.1.

```

Выбор: 1
Случайные ключи: p, q – простые из [100, 9999], d – случайное, взаимно простое с φ.
p = 4651, q = 9311, n = p·q = 43305461, φ = (p-1)(q-1) = 43291500
d = 206879 (НОД(d, φ) = 1), e = d-1 mod φ = 13175219, проверка: e·d mod φ = 1
Открытый ключ {e, n} = {13175219, 43305461}, закрытый ключ {d, n} = {206879, 43305461}

Выбор: 3
Текст (ASCII): Hello, RSA 2026!
Коды символов M: [72, 101, 108, 108, 111, 44, 32, 82, 83, 65, 32, 50, 48, 50, 54, 33]
Шифртекст C = Me mod n ({e, n} = {13175219, 43305461}):
30447288 5484913 3784637 3784637 18324008 7274101 29857261 27537519 21733373
29127549 29857261 39050216 16534966 39050216 3532151 13865455

Выбор: 4
d (Enter – текущий закрытый ключ):
n (Enter – текущий):
Шифртекст, числа через пробел (Enter – последний результат):
M = Cd mod n ({d, n} = {206879, 43305461}): [72, 101, 108, 108, 111, 44, 32, 82, 83, 65, 32, 50, 48, 50, 54, 33]
Расшифрованный текст: 'Hello, RSA 2026!'

Выбор: 6
Противник знает только открытый ключ {e, n} = {13175219, 43305461}.
Факторизация: n = 4651 · 9311, φ = 43291500, d = e-1 mod φ = 206879 (0.107 мс)
Совпадает с настоящим закрытым ключом: да

```

Рисунок 6.1 — Сеанс: ключи, шифрование, расшифрование, вычисление закрытого ключа

Расшифрование выполнено по $\{d, n\}$ без p, q , закрытый ключ восстановлен. Охват ASCII проверен отдельно: коды 0..127 и 0..255 прогнаны через `modpow` при $n = 33, 129, 265, 10403, 1005973$.

Таблица 6.4 — Доля кодов ASCII, верно проходящих цикл шифрование→расшифрование, при разных n

$p \cdot q$	n	Верно из кодов 0..127	Верно из кодов 0..255	$n \geq 256$
3·11	33	33/128	33/256	нет
3·43	129	128/128	129/256	нет
5·53	265	128/128	256/256	да
101·103	10403	128/128	256/256	да
997·1009	1005973	128/128	256/256	да

При $n = 33$ верны 33/128, при $n = 129$ — 128/128, но 129/256, при $n \geq 265$ — 256/256. Время шифрования 100 символов: наивное $(M^{**e}) \% n$ против `modpow`.

Таблица 6.5 — Время шифрования 100 символов: возведение целиком против бинарного метода по модулю

p, q	n	e	$(M^e) \% n$ целиком	<code>modpow</code>
2-значные	3233	899	1.24 мс	0.09 мс
3-значные	988027	603263	32.1 с	0.22 мс
4-значные	99400891	18877835	> 10 с на один символ (прервано)	0.23 мс

Наивное возведение заняло 1.24 мс, 32.1 с и прервано при 4-значных p, q ; `modpow` — 0.09–0.23 мс. Закрытый ключ по открытому (пункт 6, `recover_private_key`) вычислен для $n \approx 10^4, 10^6, 10^8$.

Таблица 6.6 — Вычисление закрытого ключа по открытому $\{e, n\}$ факторизацией n

n	e	Найдено $p \cdot q$	Время, мс	Восстановленный d	d совпал
10403	8743	101·103	0.004	7	да
1005973	547619	997·1009	0.023	11	да
99400891	14197279	9967·9973	0.213	7	да

Факторизация заняла 0.004–0.213 мс, d восстановлен во всех случаях. Таблицы 6.2 и 6.3 подтверждают пункт 2 задания, таблица 6.4 — пункт 3.

7 Анализ результатов

Пример методички воспроизводится только в числовом режиме. При кодах ASCII и $n = 33$ расшифрование даёт $M \bmod n$ (таблица 6.1; строки получены прямым вызовом `modpow`, поскольку `encrypt_int` отвергает $M \geq$

n). Порог $N_{\min} = 256$ взят с запасом: $n = 129$ хватает для кодов 0..127, но не для 0..255 (таблица 6.4); минимальный допустимый $n = 259$.

Валидация ключей обязательна: при составном p или $p = q$ величина $(p-1)(q-1)$ не равна $\varphi(n)$, и равенство $C^d \bmod n = M$ не гарантировано, поэтому `manual_keys` отвергает такие p, q (таблица 6.2). Условие $\text{НОД}(d, \varphi) = 1$ обеспечивает существование e : `make_keys` подбирает случайное d в цикле, неподходящее введённое d отвергается.

Таблица 6.5 показывает, что вычислять M^e целиком нельзя: 3-значные p, q — 32.1 с на 100 символов, 4-значные — более 10 с на символ; `modpow` укладывается в 0.23 мс.

Стойкость на учебных размерах отсутствует: $n \approx 10^8$ факторизуется пробным делением за 0.213 мс, и d восстанавливается из открытого ключа (таблица 6.6); реальные n — 2048–4096 бит.

Ограничения реализации: посимвольный детерминированный шифр без паддинга — одинаковые символы дают одинаковые C (таблица 6.3, текст «aaa bbb»), что допускает частотный анализ.

8 Ответы на контрольные вопросы

Чем отличаются симметричные и асимметричные криптосистемы? К какой относится RSA? В симметричной системе обе стороны используют один секретный ключ, который приходится передавать по защищённому каналу. В асимметричной системе ключей два: открытый $\{e, n\}$ публикуется, закрытый $\{d, n\}$ хранится у владельца; стойкость опирается на трудную математическую задачу. RSA — асимметричная криптосистема, основанная на трудности факторизации n . На практике применяют гибрид: RSA или протокол Диффи–Хеллмана (LP1) передаёт сеансовый ключ, а данные шифрует симметричный шифр.

Можно ли вычислить закрытый ключ, зная открытый? Формально да: разложив n на множители p и q , противник вычисляет $\varphi = (p-1)(q-1)$ и $d = e^{-1} \bmod \varphi$. Так действует пункт меню 6 (`recover_private_key`): учебные n

до 10^8 факторизуются за доли миллисекунды, d восстанавливается точно (таблица 6.6). Однако эффективного алгоритма факторизации не известно, и для применяемых на практике n длиной 2048 бит такое вычисление невыполнимо. Стойкость RSA определяется только размером n .

Может ли алгоритм RSA использоваться в электронной цифровой подписи? Да, RSA — один из стандартных алгоритмов подписи; роли ключей меняются местами. Отправитель вычисляет хэш сообщения $H(M)$ и подписывает его закрытым ключом: $S = H(M)^d \bmod n$; получатель открытым ключом проверяет равенство $S^e \bmod n = H(M)$. Подписывается хэш, а не само сообщение: объём вычислений не зависит от длины M . Значение хэша должно быть меньше n — то же правило $M < n$, что и при шифровании. Подпись обеспечивает целостность, аутентичность и неотказуемость (задание ЛР4).

9 Выводы

Разработана программа `rsa.py`: случайная генерация ключей, ручной ввод p , q , d , шифрование, расшифрование по $\{d, n\}$, самопроверка, вычисление закрытого ключа по открытому. Воспроизведён пример методички ($3, 1, 2 \rightarrow 9, 1, 29$); при $n \geq 256$ цикл проходят 256/256 кодов; из восьми случаев ручного ввода два корректных набора приняты, шесть некорректных отвергнуты с сообщением; четыре текста при случайных ключах восстановлены верно.

Корректность требует: простые $p \neq q$, $n > \max M$, $\text{НОД}(d, \varphi) = 1$, быстрое возведение в степень по модулю (`modpow`). Учебные ключи с четырёхзначными p , q стойкости не имеют: `recover_private_key` факторизует n и находит d за доли миллисекунды.

Приложение А. Листинг программы

Файл `rsa.py`, Python 3, только стандартная библиотека. Запуск:

`python3 rsa.py.`

```
#!/usr/bin/env python3
"""Лабораторная работа №2. Алгоритм шифрования RSA.

Ключи по методичке:  $p, q$  — простые;  $n = p \cdot q$ ;  $\phi = (p-1)(q-1)$ ;  $d$  выбирается
случайно и взаимно просто с  $\phi$ ;  $e$  находится из условия  $e \cdot d \bmod \phi = 1$ .
Открытый ключ  $\{e, n\}$ , закрытый  $\{d, n\}$ .
Шифрование посимвольно:  $M$  = код символа,  $C = M^e \bmod n$ ,  $M = C^d \bmod n$ .
Каждый блок должен быть меньше  $n$ . Порог  $n \geq 256$  взят с запасом: любой байт
0..255 меньше  $n$ ; для 7-битного ASCII (0..127), который принимает программа,
хватило бы  $n \geq 128$ .

Режимы: случайные ключи, ручной ввод ключей, шифрование, расшифрование,
самопроверка (пример методички + прогон всех кодов ASCII), вычисление
закрытого ключа по открытому (факторизация  $n$  — к контрольному вопросу 2).

Только стандартная библиотека. Запуск: python3 rsa.py
"""

import random
import time

P_MIN, P_MAX = 100, 9999          # диапазон случайных простых  $p, q$ 
N_MIN = 256                       #  $n \geq 256$  — любой байт 0..255 меньше  $n$  (для ASCII 0..127
хватило бы 128)

# Пример методички:  $p = 3, q = 11, n = 33, d = 3, e = 7$ ; "CAB" при  $A=1, B=2, C=3$ 
EXAMPLE = {"p": 3, "q": 11, "d": 3, "e": 7, "M": [3, 1, 2], "C": [9, 1, 29]}

# --- Арифметика -----
def modpow(base, exp, mod):
    """Бинарное возведение в степень по модулю; остаток после каждого умножения.
    Считать  $base^{**}exp$  целиком нельзя: при 4-значных  $p, q$  это число из миллионов
    цифр. Эквивалентно встроенному pow(base, exp, mod)."""
    if mod == 1:
        return 0
    result, base = 1, base % mod
    while exp > 0:
        if exp & 1:
            result = result * base % mod
        base = base * base % mod
        exp >>= 1
    return result

def is_prime(n):
    """Пробное деление до  $\sqrt{n}$ : для  $n < 10^6$  мгновенно."""
    if n < 2:
        return False
    if n % 2 == 0:
        return n == 2
    k = 3
    while k * k <= n:
        if n % k == 0:
            return False
        k += 2
    return True
```

```

def gen_prime(lo=P_MIN, hi=P_MAX):
    while True:
        n = random.randint(lo, hi)
        if is_prime(n):
            return n

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def egcd(a, b):
    """Расширенный алгоритм Евклида: возвращает (g, x, y), где  $a \cdot x + b \cdot y = g = \text{НОД}(a, b)$ ."""
    x0, x1, y0, y1 = 1, 0, 0, 1
    while b:
        q, a, b = a // b, b, a % b
        x0, x1 = x1, x0 - q * x1
        y0, y1 = y1, y0 - q * y1
    return a, x0, y0

def modinv(a, m):
    """Обратный элемент  $a^{-1} \bmod m$ ; существует только при  $\text{НОД}(a, m) = 1$ .
    Результат нормализуется в диапазон  $[1, m-1]$ . Эквивалентно row(a, -1, m)."""
    g, x, _ = egcd(a, m)
    if g != 1:
        raise ValueError(f"обратного элемента нет:  $\text{НОД}(\{a\}, \{m\}) = \{g\}$ ")
    return x % m

# --- Ключи -----
def key_errors(p, q, d=None):
    """Список нарушений условий на p, q, d (пустой список — всё в порядке)."""
    errors = []
    if not is_prime(p):
        errors.append(f"p = {p} не простое")
    if not is_prime(q):
        errors.append(f"q = {q} не простое")
    if p == q:
        errors.append("p и q должны быть различны: при p = q формула  $\phi = (p-1)(q-1)$  неверна")
    if p * q < N_MIN:
        errors.append(f"n = p * q = {p * q} < {N_MIN}: не любой код символа поместится в блок M < n")
    if d is not None and not errors:
        phi = (p - 1) * (q - 1)
        if not 1 < d < phi:
            errors.append(f"d должно лежать в (1,  $\phi$ ) = (1, {phi})")
        elif gcd(d, phi) != 1:
            errors.append(f"НОД(d,  $\phi$ ) = НОД({d}, {phi}) = {gcd(d, phi)} ≠ 1: обратного e нет")
    return errors

def make_keys(p, q, d=None, check=True):
    """Генерация пары ключей по методичке. Возвращает ((e, n), (d, n),  $\phi$ ).
    Если d не задано — выбирается случайно из (1,  $\phi$ ) до выполнения  $\text{НОД}(d, \phi) = 1$ .
    check=False отключает проверку условий (нужно для примера методички с n = 33 < 256)."""

```

```

Корректность  $M = C^d \bmod n$  для всех  $0 \leq M < n$  следует из  $e \cdot d \equiv 1 \pmod{\phi}$  и теоремы Эйлера (при  $\text{НОД}(M, n) = 1$ ), а для  $M$ , кратных  $p$  или  $q$ , — из китайской теоремы об остатках; поэтому  $p \neq q$  обязательно: при  $p = q$  значение  $\phi$  иное."""
if check:
    errors = key_errors(p, q, d)
    if errors:
        raise ValueError("; ".join(errors))
n, phi = p * q, (p - 1) * (q - 1)
if d is None:
    while True:
        d = random.randint(2, phi - 1)
        if gcd(d, phi) == 1:
            break
e = modinv(d, phi)
return (e, n), (d, n), phi

# --- Шифрование -----
def encrypt_int(m, e, n):
    if not 0 <= m < n:
        raise ValueError(f"блок M = {m} должен лежать в [0, n-1] = [0, {n - 1}]")
    return modpow(m, e, n)

def decrypt_int(c, d, n):
    return modpow(c, d, n)

def encrypt_text(text, e, n):
    """Каждый символ — отдельный блок M = ord(символа). Возвращает список чисел."""
    return [encrypt_int(ord(ch), e, n) for ch in text]

def decrypt_text(numbers, d, n):
    """Обратное преобразование. При неверном ключе или чужом шифртексте блок M может не быть кодом символа — тогда ValueError с понятным сообщением."""
    chars = []
    for c in numbers:
        m = decrypt_int(c, d, n)
        if m > 0x10FFFF:
            raise ValueError(f"блок M = {m} не является кодом символа: неверный ключ или чужой шифртекст")
        chars.append(chr(m))
    return "".join(chars)

# --- Самопроверка и атака -----
def self_test(verbose=True):
    """1) пример методички (p=3, q=11, d=3 → e=7; 3,1,2 → 9,1,29 → 3,1,2);
    2) прогон всех кодов 0..255 через ключ с n ≥ 256 (p=5, q=53, n=265)."""
    ex = EXAMPLE
    (e, n), (d, _), phi = make_keys(ex["p"], ex["q"], ex["d"], check=False)
    c = [encrypt_int(m, e, n) for m in ex["M"]]
    back = [decrypt_int(x, d, n) for x in c]
    ok1 = e == ex["e"] and c == ex["C"] and back == ex["M"]
    if verbose:
        print(f" Пример методички: p={ex['p']}, q={ex['q']}, n={n}, φ={phi}, d={d} → e={e} (ожидалось {ex['e']})")
        print(f" M = {ex['M']} → C = {c} (ожидалось {ex['C']}) → M' = {back}: {'PASS' if ok1 else 'FAIL'}")
    (e, n), (d, _), _ = make_keys(5, 53, 7)
    matched = sum(decrypt_int(encrypt_int(m, e, n), d, n) == m for m in range(256))
    ok2 = matched == 256
    if verbose:

```

```

        print(f" Все коды 0..255 через ключ n={n} (p=5, q=53, d={d}, e={e}): совпало
{matched}/256: {'PASS' if ok2 else 'FAIL'}")
        print(f" Итог самопроверки: {'OK' if ok1 and ok2 else 'ОШИБКА'}")
        return ok1 and ok2

def factor(n):
    """Разложение n = p·q пробным делением до sqrt(n). Возвращает (p, q) или None."""
    if n % 2 == 0:
        return 2, n // 2
    k = 3
    while k * k <= n:
        if n % k == 0:
            return k, n // k
        k += 2
    return None

def recover_private_key(e, n):
    """Действия противника, знающего только открытый ключ {e, n}:
    факторизовать n → φ → d = e(-1) mod φ. Возвращает (p, q, d, время в мс)."""
    t0 = time.perf_counter()
    p, q = factor(n)
    phi = (p - 1) * (q - 1)
    d = modinv(e, phi)
    return p, q, d, (time.perf_counter() - t0) * 1000

# --- Ввод -----
def read_int(prompt, check=None, allow_empty=False):
    """Читает целое; check(value) возвращает None (OK) или текст ошибки.
    При allow_empty пустая строка возвращает None."""
    while True:
        raw = input(prompt).strip()
        if allow_empty and raw == "":
            return None
        try:
            value = int(raw)
        except ValueError:
            print(" Нужно ввести целое число.")
            continue
        error = check(value) if check else None
        if error is None:
            return value
        print(" ", error + ".")

def read_text(prompt):
    """Строка из символов ASCII (коды 0..127); иначе — сообщение и повтор."""
    while True:
        text = input(prompt)
        bad = [ch for ch in text if ord(ch) > 127]
        if not bad:
            return text
        print(f" Символ «{bad[0]}» (код {ord(bad[0])}) вне кодировки ASCII, повторите
ввод.")

class State:
    public = None          # (e, n)
    private = None         # (d, n)
    phi = None
    last_cipher = None     # последний шифртекст (список чисел)

```

```

def show_keys(st, p=None, q=None):
    (e, n), (d, _) = st.public, st.private
    if p is not None:
        print(f" p = {p}, q = {q}, n = p·q = {n}, φ = (p-1)(q-1) = {st.phi}")
        print(f" d = {d} (НОД(d, φ) = {gcd(d, st.phi)}), e = d-1 mod φ = {e}, проверка: e·d mod φ = {e * d % st.phi}")
        print(f" Открытый ключ {{e, n}} = {{{e}, {n}}}, закрытый ключ {{d, n}} = {{{d}, {n}}}")

def random_keys(st):
    print(f"Случайные ключи: p, q — простые из [{P_MIN}, {P_MAX}], d — случайное, взаимно простое с φ.")
    p = gen_prime()
    q = gen_prime()
    while q == p:
        q = gen_prime()
    st.public, st.private, st.phi = make_keys(p, q)
    show_keys(st, p, q)

def manual_keys(st):
    print("Ручной ввод ключей.")
    p = read_int(" p (простое): ", lambda v: None if is_prime(v) else f"p = {v} не простое")

    def check_q(v):
        if not is_prime(v):
            return f"q = {v} не простое"
        if v == p:
            return "q должно отличаться от p: при p = q формула φ = (p-1)(q-1) неверна"
        if p * v < N_MIN:
            return f"n = p·q = {p * v} < {N_MIN}: не любой код символа поместится в блок M < n"
        return None

    q = read_int(" q (простое, q ≠ p, p·q ≥ 256): ", check_q)
    phi = (p - 1) * (q - 1)
    print(f" n = {p * q}, φ = {phi}")

    def check_d(v):
        if not 1 < v < phi:
            return f"d должно лежать в (1, φ) = (1, {phi})"
        if gcd(v, phi) != 1:
            return f"НОД(d, φ) = {gcd(v, phi)} ≠ 1: обратного e нет"
        return None

    d = read_int(" d (1 < d < φ, НОД(d, φ) = 1): ", check_d)
    st.public, st.private, st.phi = make_keys(p, q, d)
    show_keys(st, p, q)

def encrypt_mode(st):
    if st.public is None:
        print(" Сначала задайте ключи (пункт 1 или 2).")
        return
    e, n = st.public
    text = read_text(" Текст (ASCII): ")
    st.last_cipher = encrypt_text(text, e, n)
    print(f" Коды символов M: {[ord(ch) for ch in text]}")
    print(f" Шифртекст C = M^e mod n ({{e, n}} = {{{e}, {n}}}):")
    print(" " + " ".join(map(str, st.last_cipher)))

```

```

def decrypt_mode(st):
    has_key = st.private is not None
    d = read_int(" d (Enter — текущий закрытый ключ): " if has_key else " d: ",
                  lambda v: None if v > 0 else "d должно быть положительным",
                  allow_empty=has_key)
    n = read_int(" n (Enter — текущий): " if has_key else " n: ",
                  lambda v: None if v > 1 else "n должно быть больше 1",
                  allow_empty=has_key)
    if d is None:
        d, n = st.private
    elif n is None:
        n = st.private[1]
    prompt = " Шифртекст, числа через пробел" + (" (Enter — последний результат): "
if st.last_cipher else ": ")
    while True:
        raw = input(prompt).strip()
        if raw == "" and st.last_cipher:
            numbers = st.last_cipher
            break
        try:
            numbers = [int(x) for x in raw.split()]
        except ValueError:
            print(" Нужны целые числа через пробел.")
            continue
        if any(not 0 <= c < n for c in numbers):
            print(f" Каждое число должно лежать в [0, n-1] = [0, {n - 1}].")
            continue
        break
    try:
        text = decrypt_text(numbers, d, n)
    except ValueError as err:
        print(f" Не удалось расшифровать: {err}.")
        return
    print(f" M = C^d mod n ({d, n} = {{d, n}}): {[ord(ch) for ch in text]}")
    print(f" Расшифрованный текст: {text!r}")

def attack_mode(st):
    if st.public is None:
        print(" Сначала задайте ключи (пункт 1 или 2).")
        return
    e, n = st.public
    print(f" Противник знает только открытый ключ {{e, n}} = {{e, n}}.")
    p, q, d, ms = recover_private_key(e, n)
    print(f" Факторизация: n = {p} · {q}, φ = {(p - 1) * (q - 1)}, d = e⁻¹ mod φ = {d} ({ms:.3f} mc)")
    print(f" Совпадает с настоящим закрытым ключом: {'да' if d == st.private[0] else 'нет'}")

def main():
    st = State()
    menu = ("Лабораторная работа №2. Алгоритм шифрования RSA\n"
            " 1 — случайные ключи\n"
            " 2 — ввод ключей вручную (p, q, d)\n"
            " 3 — зашифровать текст открытым ключом {e, n}\n"
            " 4 — расшифровать закрытым ключом {d, n}\n"
            " 5 — самопроверка (пример методички, все коды ASCII)\n"
            " 6 — вычислить закрытый ключ по открытому (факторизация n)\n"
            " 0 — выход")
    actions = {"1": random_keys, "2": manual_keys, "3": encrypt_mode, "4":
decrypt_mode,
            "5": lambda _: self_test(), "6": attack_mode}
    try:
        while True:

```

```
        print(menu)
        choice = input("Выбор: ").strip()
        if choice == "0":
            break
        if choice in actions:
            actions[choice](st)
        else:
            print(" Нет такого пункта меню.")
    except (EOFError, KeyboardInterrupt):
        print("\nВыход.")

if __name__ == "__main__":
    main()
```